Simultaneous Estimation and Control with MCTS

Preston Culbertson and Patrick Slade

***Abstract*— Robots performing manipulation tasks have to operate under considerable uncertainty about both their pose and the dynamics of their payload. In order to remain robust to modeling error, and shifts in payload dynamics, agents must simultaneously perform estimation and control tasks. However, the optimal estimation actions are often not the optimal actions for accomplishing the control tasks, and thus agents must trade off between "exploration" actions that improve their payload model, and "exploiting" this model to accomplish the control task. This work poses this problem as a Belief-Space Markov Decision Process (BS-MDP), using an Extended Kalman Filter (EKF) to model the transitions between belief states, and a reward based on the agent's estimated distance to goal, control effort, and uncertainty about model parameters. It then uses a Monte-Carlo Tree Search (MCTS) to select control actions that nearly optimally balance reducing model uncertainty and reaching the goal state. This method outperforms a naive Linear-Quadratic-Gaussian (LQG) controller by nearly 80 percent in a simple control task, and is broadly applicable to a variety of dynamics models and reward functions.**

I. INTRODUCTION

A. Motivation

The "dual control" problem is a class of control problems where the system dynamics are initially unknown. While the robots may have explicit dynamics models derived from the equations of motion, they often must perform parameter estimation tasks to fit these models to an unknown object.

This fitting can be performed online by using methods such as Kalman filtering or least-squares regression, but the control actions that optimally reveal the payload's dynamics are usually not optimal for reaching some goal state or minimizing the control effort. In practice, this is often solved by having the agent first perform an "estimation" task, where it takes control actions that reveal the object's dynamics, and then perform the control task, this strategy is clearly non-optimal, since the agent could potentially use its estimation actions to begin performing the control task.

Thus, a robot manipulating a previously unknown payload faces an exploration-exploitation problem: while some actions would allow it to better characterize its object (and explore the state space), these actions are often different from the optimal control policy that would be formulated with an explicit object model. Optimal controllers for robots performing uncertain manipulation tasks must balance these two objectives, estimation and control.

Solving this problem is especially important for extending the flexibility and robustness of real-world robotic systems, as payload dynamics models often contain uncertainty and error that a static, non-estimating, controller would be unable to respond to. Further, since payload dynamics may shift during the manipulation task (e.g. change of mass or inertial properties, shift in friction/damping models), a controller performing online estimation will be able to respond in real time to these shifts.

B. Related Work

The dual control problem was first posed by Feldbaum in 1960, and he proved that an optimal solution could be found via dynamic programming [1]. However, since the state, action, and belief spaces are continuous, the exact solution is intractable. Thus, various approximate solutions have been sought. The problem has been approached via a variety of methods, including adaptive control, sliding mode control, and stochastic optimal control. Filatov's work gives an extensive survey of existing work [2]. Many of these methods are able to successfully see convergence in the payload model to the true values, as well as to control the payload to a goal state. Many of these methods are grounded in control theory, and so they are able to make good guarantees on the system stability, while making no claims to optimality.

On the other hand, due to the recent rise in computation power and algorithmic developments, approximate solutions to Partially Observable Markov Decision Processes (POMDPs) with moderate state spaces have become tractable. POMDP-based approaches have been applied to the dual control problem, which uses a QMDP-based heuristic for an offline tree search to determine near-optimal actions [3]. However, this method requires discretizing the state space in order to solve the POMDP, which makes it hard to scale, especially for complex state space models such as 3-D manipulation tasks. Further, since this policy is calculated offline it cannot respond quickly to large shifts in payload dynamics.

This work builds off an online method previously implemented in [4] that used a value-iteration-based solution for Gaussian POMDPs, and involves re-planning control sequences along a predefined trajectory as an EKF updates the belief over the model parameters. While this previous method does prove the feasibility of online estimation and control with POMDPs, it does not benchmark its performance relative to other naive or static controllers, and it does not state the algorithm run-time, to determine its applicability to real-time dynamical systems.

II. BACKGROUND

In this section, we will briefly review some of the techniques underlying our work including BS-MDPs, MCTS, and EKFs.

A. MDPs, POMDPs, and BS-MDPs

A Markov Decision Process is a mathematical framework for a sequential decision-process in which an agent will move (in general) stochastically between states over time, accruing various rewards for entering certain states, but may affect their trajectory and rewards by taking actions at each time step. A MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where:

- $\mathcal{S}$ is the set of states the system may reach,
- $\mathcal{A}$ is the set of actions the agent may take,
- $T(s' \mid s, a)$ is the probability of transitioning to state s' by taking action a at state s,
- $R(s, a)$ is the reward (or cost) of taking action a at state s, and
- $\gamma \in [0, 1]$ is the discount factor for future rewards.

"Solving" an MDP involves developing a "policy", $\pi(s) : \mathcal{S} \rightarrow \mathcal{A}$, which maps each state to an optimal action that will accrue the most rewards over some planning horizon.

For infinite-horizon, discounted MDPs, the optimal policy has been shown to satisfy the *Bellman Equations*:

$$\pi^*(s) = \underset{a}{\operatorname{argmax}}\Big(R(s, a) + \gamma * \sum_{s'} T(s' \mid s, a) U^*(s')\Big) \quad (1)$$

where

$$U^*(s) = \max_{a}\Big(R(s, a) + \gamma * \sum_{s'} T(s' \mid s, a) U^*(s')\Big) \quad (2)$$

For small, discrete state and action spaces, the Bellman Equations may be solved explicitly with Dynamic Programming (DP). However, for problems with large or continuous state and action spaces, only approximate solutions may be found via Approximate Dynamic Programming (ADP). Below, we will discuss *Monte Carlo Tree Search*, an online ADP algorithm we applied to solve our BS-MDP.

A POMDP is an extension of an MDP, where the agent cannot directly observe its true state, but may only take observations which are stochastically dependent on this state. A POMDP thus contains a model of the probability of taking a certain observation at a certain state. The agent then forms a belief state, $b(s)$ which gives the probability of being in a certain state s. The agent updates its belief at each step, depending on its previous action, the reward received, and the observation it took. Since the agent may hold any combination of beliefs about its location in the state space, the belief space $\mathcal{B}$ is continuous, making POMDPs computationally intensive to solve.

POMDPs may be recast as a Belief-State MDP (BS-MDP), where the state space is composed of the belief space $\mathcal{B}$. Thus, the agent's rewards are dependent on its current belief and action, and it transitions between belief states by taking actions. Since $\mathcal{B}$ is continuous, BS-MDPs must be solved approximately using ADP. Posing a POMDP as a BS-MDP allows MDP solution methods to be applied to partially-observable decision problems.

B. Monte-Carlo Tree Search

MCTS is a sampling-based online approach to approximately solving MDPs. MCTS requires a generative model G which can draw a possible state and reward $(s', r) \sim G(s, a)$. The algorithm performs a forward search through the state space, using G to draw prospective trajectories and rewards.

In general, MCTS involves four stages:

- *Selection*: Traverse the tree to a leaf node of choice, storing the accrued rewards and number of times each node is visited.
- *Expansion*: If at a leaf node, take a new action, and add the resulting new state to the tree.
- *Rollout*: At each leaf node, perform a "roll out" simulation according to some pre-determined (or random) policy, to estimate this state's value.
- *Propagation*: Propagate the new state's estimated values along the taken path.

Our implementation of MCTS uses Double Progressive Widening (see [5]) to balance exploration and exploitation in continuous action spaces.

C. Extended Kalman Filter

In problems with linear Gaussian dynamics and observation functions, it has been shown that the optimal observer is a Kalman Filter. A Kalman Filter is an iterative algorithm that can exactly update Gaussian beliefs over the state, given the action taken, the observation received, and the transition/observation models.

For systems with nonlinear dynamics, an Extended Kalman Filter may be used to near-optimally update the beliefs with each step. Let x_k, a_k, and o_k be the state, action, and observation at time $t = k$. For a system with nonlinear-Gaussian dynamics, the transition and observation models can be expressed as

$$x_{k+1} = f(x_k, a_k) + v_k \quad (3)$$

$$o_k = h(x_k, a_k) + w_k \quad (4)$$

where f, h are (in general) nonlinear functions, $v \sim \mathcal{N}(0, Q)$, and $w \sim \mathcal{N}(0, R)$. The belief over the current state may be represented as a multivariate-Gaussian,

$$b(x_k) \sim \mathcal{N}(\hat{x}_k, \Sigma) \quad (5)$$

Using an EKF, the belief is updated at each timestep by first predicting a new state and covariance based on the action taken, i.e.

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, a_{k-1}) \quad (6)$$

$$\Sigma_{k|k-1} = F_{k-1} \Sigma_{k-1|k-1} F_{k-1}^T + Q_{k-1} \quad (7)$$

where F is the linearized dynamics about the current belief,

$$F_{k-1} = \left.\frac{\partial f}{\partial x}\right|_{\hat{x}_{k-1|k-1}, u_{k-1}} \tag{8}$$

Then a residual between the observation and the predicted state is calculated:

$$r_k = o_k - h(\hat{x}_{k|k-1}, a_{k-1}) \tag{9}$$

The residual covariance, S_k and near-optimal Kalman gain, K_k are then calculated according to

$$S_k = H_k P_{k|k-1} H_k^T + R_k \tag{10}$$

$$K_k = P_{k|k-1} H_k^T S_k^{-1} \tag{11}$$

where H_k is the linearized observation function,

$$H_k = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_{k|k-1}, u_{k-1}} \tag{12}$$

Finally, the belief updated is completed according to

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k r_k \tag{13}$$

$$\Sigma_{k|k} = (I - K_k H_k) P_{k|k-1} \tag{14}$$

Due to the linearization of the dynamics and observation models, EKFs are not proven to converge, nor are they proven to be optimal observers. However, in practice they work quite well for observing systems with nonlinear-Gaussian dynamics.

III. Problem Formulation

A. Transition and Observation Model

Let us consider a robot trying to control a system with linear-Gaussian dynamics. The system may be described by

$$x_{k+1} = f(x_k, u_k; p) + v_k \tag{15}$$

where x_k, u_k are the state and control action at time $t = k$, the process noise $v_k \sim \mathcal{N}(0, Q)$, and f is a time-invariant function given by

$$f(x_k, u_k; p) = A(p)x_k + B(p)u_k \tag{16}$$

The observation model is described by the linear equation

$$o_k = h(x_k, u_k; p) + w_k \tag{17}$$

where o_k is the observation taken at time $t = k$, the measurement noise $w_k \sim \mathcal{N}(0, R)$, and h is a time-invariant linear function given by

$$h(x_k, u_k; p) = C(p)x_k + D(p)u_k \tag{18}$$

However, while f and h can often be determined *a priori*, according to physical equations, the parameters p cannot be known beforehand. The parameters can thus be appended to the state vector to form a state-parameter vector,

$$s_k = \begin{bmatrix} x_k \\ p \end{bmatrix} \tag{19}$$

Thus, the system dynamics for s_k may be described by the nonlinear equations

$$s_{k+1} = \begin{bmatrix} A(p) & 0 \\ 0 & I \end{bmatrix} s_k + \begin{bmatrix} B(p) \\ 0 \end{bmatrix} u_k + \tilde{v}_k \tag{20}$$

where $\tilde{v} \sim \mathcal{N}(0, \mathrm{diag}(Q, P)$ and P is a parameter drift matrix.

The observation model is described by

$$o_k = \begin{bmatrix} C(p) & 0 \end{bmatrix} s_k + D(p)u_k + w_k \tag{21}$$

B. Belief-State MDP

If we use an EKF to describe our belief about the current state in the state-parameter space, we can then form a BS-MDP over all possible EKF states. This BS-MDP is described by the tuple $(\mathcal{S}, \mathcal{A}, T, R)$, where:

- $\mathcal{S}$ is the space of all possible beliefs, where our state $b(s_k) \sim \mathcal{N}(\hat{s}_k, \Sigma_k)$,
- $\mathcal{A}$ is all possible actions that the agent may take (e.g. forces, torques)
- $T(b' \mid b, a)$ describes a distribution over the possible EKF states after a belief update. Since this distribution depends on the observation model, it is a complex distribution.
- $R(b, a)$ is a reward function for a given belief and action. It can be constructed as desired for a given control task, but, in our work, decomposes into $R(\hat{s}_k, a) + R(\Sigma_k)$, where $R(\hat{s}_k, a)$ is a quadratic reward for the estimated state and action, and $R(\Sigma_k)$ is a reward dependent on the EKF's uncertainty.

Solving this BS-MDP gives a policy that optimally maximizes the sum of expected rewards over some planning horizon.

IV. Approach

In order to solve this BS-MDP problem we outline how a nonlinear state-space model can be near-optimally solved with an EKF and MCTS.

The purpose of this method is to produce a control policy for execution. The dynamics of a linear or nonlinear system can be described using state-space models with control inputs following (15). Any system described this way with an initial belief state can run MCTS to find an approximately optimal action from a policy with a given depth and number of operations.

Taking this action propagates the dynamics state, which will be partially observed. This observation and action will be used to update the belief state with the EKF, improving our estimates about the parameters. This is written out in Algorithm 1

Input: $b_0(s), s_0$
for $t \in$ *[0, T)* **do**
  $u_t \leftarrow MCTS(b_t)$
  $s_{t+1} \leftarrow DynamicsPropagate(s_t, u_t)$
  $o_{t+1} \leftarrow ReceiveMeasurements(s_{t+1})$
  $b_{t+1} \leftarrow EKF(b_t, u_t, o_{t+1})$
end
return $u_i|_{i=0}^{T}$

Algorithm 1: Pseudocode to implement our method of simultaneous estimation and control with MCTS

V. Simulation and Results

A double integrator model was used to test our approach with MCTS. A classic linear-quadratic-Gaussian (LQG) controller, an EKF in conjunction with a linear-quadratic-regulator, was used as a basic optimal control benchmark. The LQG controller first estimates the mass of the object using random forces until an uncertainty threshold is met and then executes an optimal control strategy. Although widely used, the optimality of LQG doesn't ensure robustness which causes difficulty in situations with large noise or uncertainty [6]. The roll-out used in out MCTS used a position controller to select a force proportional to the distance from the goal state.

A. Double Integrator Model

We first considered the simple control problem of an agent that can apply a force to a point mass moving in one dimension. The dynamics of this system may be described as

$$x_{k+1} = \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} x_k + \begin{bmatrix} \frac{\Delta t}{m} \\ (\frac{\Delta t}{m})^2 \end{bmatrix} f_k + v_k \tag{22}$$

$$o_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} x_k + w_k \tag{23}$$

where process noise $v_k \sim \mathcal{N}(0, Q)$, measurement noise $w_k \sim \mathcal{N}(0, R)$, and f_k is the force applied at time $t = k$.

B. Implementation Details

When implementing our method, real-world constraints and conditions were considered but not addressed. The initial conditions were chosen without any prior information from a motion or path planning algorithm. Although in real systems constraints such as control saturation must be accounted for, these simulations allowed any control input that could be sampled from a normal distribution with a mean of 0 and standard deviation of 100N. The reward function did implement costs for control inputs, but it didn't bound them.

An existing solver for MCTS with DPW was used from the POMDPs.jl repository [7]. This was used with the EKF transition model as previously described for state-propagation of 0.1 second time intervals. Each complete MCTS and EKF update comprised an average run-time of 0.0974 seconds using approximately 15% CPU processing on an Intel i7. This provides evidence that our method could be implemented for real-time experimental use.

C. Results

The simultaneous estimation of the point mass and control was simulated with 30 trials for each of 3 cases with different initial beliefs of mass of 18kg, 12kg, and 8kg with the same reward cost functions for both controllers. The LQG mass estimation in Figure 1 shows the noisy walk the belief takes before converging in roughly 80 time steps. The MCTS mass estimation in Figure 2 shows a less random walk while converging much more quickly, on the order of four times faster. The highlighted regions show the 98% confidence intervals, which correspond closely between both methods, as do the standard deviations.

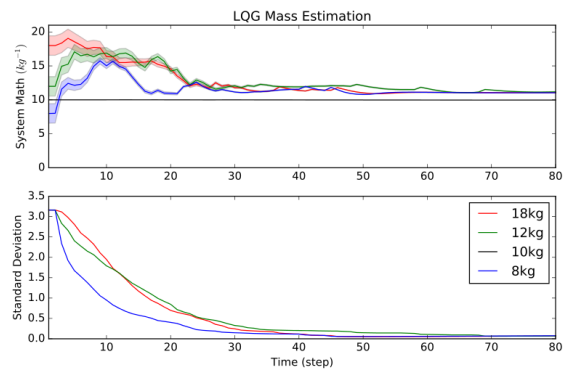

Fig. 1. Mass estimation and standard deviation of LQG controller

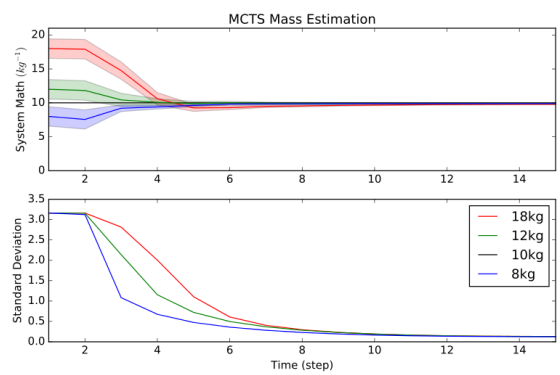

Fig. 2. Mass estimation and standard deviation of MCTS controller

The LQG average profile of position, velocity, and forces applied during the simulation is displayed in Figure 3 which highlights the initial random forces, close to 0, used to minimize the uncertainty of the mass. During this period the LQG controller is unable to move towards the goal position, because of this we see it drift away from the goal position due to an initial velocity. The LQG then executes smooth position control, albeit with a slightly rough force control due to no control input restrictions that causes some oscillation in the velocity.

The MCTS profiles shown in Figure 4 show the immediately large negative force applied to reduce uncertainty about the mass while simultaneously moving towards the

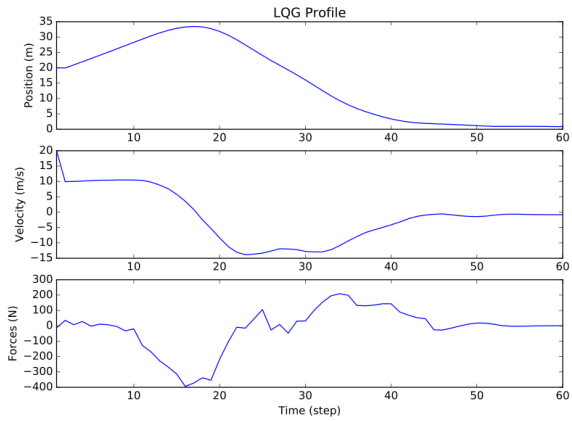

Fig. 3. Position, velocity, and force profiles of LQG controller

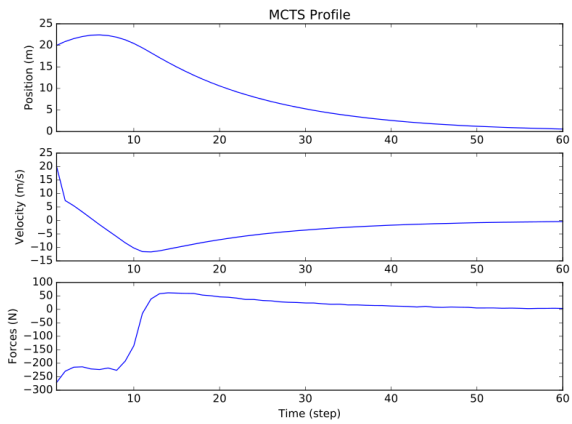

Fig. 4. Position, velocity, and force profiles of MCTS controller

goal position. Once the LQG finishes estimating the mass it takes a very similar force trajectory to MCTS, but due to the optimality the magnitude of these forces is much higher.

The reward costs for MCTS for the 18kg, 12kg, and 8kg tests were 19.1%, 17.7%, and 28.0% of the costs for LQG, respectively. A 5.4% difference is found between the maximum and minimum costs for the MCTS while LQG had a 50.2% difference for the same tests, exemplifying the ability of MCTS to quickly estimate the mass while proceeding to the goal to reduce the cost.

VI. Conclusions and Future Work

In this paper, we proposed posing a robot performing a manipulation task for a payload of unknown dynamics as a BS-MDP. By appending the model parameters to the state vector of a known, parameterized transition model, we formed a nonlinear dynamics equation for the state-parameter space, and used an EKF to update our beliefs about our location in this space. We then proposed using MCTS to generate an online, near-optimal policy at each timestep of the manipulation task.

From the simulation results, our MCTS-based method clearly outperformed a naive LQG-based control strategy, and shows promise for wide applicability to other control problems. The MCTS was able to compute in real time, and thus should be suitable for implementation on real systems. However, there are many ways in which our method could be improved.

For future work, we plan to test our algorithm on more complex dynamical systems with additional unknown parameters. Further, we would like to test our controller's performance against other adaptive control methods. We would also like to use an offline solution method, if possible, so that execution onboard a robotic agent would be faster than performing a MCTS at each timestep. We plan to investigate using Monte Carlo Value Iteration (MCVI), which has shown promise for solving continuous-state, continuous-action POMDPs. We would also like to generalize these results to a multi-agent context if possible, to allow multiple robots to collaboratively move unknown payloads.

VII. Acknowledgements

We would like to thank Zach Sunberg for his invaluable help and advice, especially on implementing MCTS. We would also like to thank Mykel Kochenderfer and the AA228 course staff for sharing their passion for POMDPs.

References

[1] A. Feldbaum, "Dual control theory. i," *Avtomatika i Telemekhanika*, vol. 21, no. 9, pp. 1240–1249, 1960.

[2] N. M. Filatov and H. Unbehauen, "Survey of adaptive dual control methods," *IEE Proceedings-Control Theory and Applications*, vol. 147, no. 1, pp. 118–128, 2000.

[3] H. Bai, D. Hsu, and W. S. Lee, "Planning how to learn," in *Robotics and Automation (ICRA), 2013 IEEE International Conference on*, pp. 2853–2859, IEEE, 2013.

[4] D. J. Webb, K. L. Crandall, and J. van den Berg, "Online parameter estimation via real-time replanning of continuous gaussian pomdps," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5998–6005, IEEE, 2014.

[5] D. Brand and S. Kroon, "Sample evaluation for action selection in monte carlo tree search," in *Proceedings of the Southern African Institute for Computer Scientist and Information Technologists Annual Conference 2014 on SAICSIT 2014 Empowered by Technology*, p. 314, ACM, 2014.

[6] M. Green and D. J. Limebeer, *Linear robust control*. Courier Corporation, 2012.

[7] "Pomdps.jl," in *http://juliapomdp.github.io/POMDPs.jl/latest/*, Stanford Intelligent Systems Laboratory.